Foundational Notes on RNN and GRU

From Naive Sequential Memory to Gated Sequence Modeling

April 6, 2026

Contents

1 Why Sequence Memory Matters

Many machine learning models are comfortable when every input can be treated as a fixed bag of numbers. A house price predictor may take area, age, and location all at once. A standard classifier may examine one image and then stop. Sequence problems are different. Meaning now depends on order, and order means the model must remember.

Consider the short sentence

``the movie is not good''.

If one only looks at the last word `good`, the sentence appears positive. The true meaning depends on something that happened earlier, namely the word `not`. The practical problem is therefore simple to state:

> How can a model read a sequence one step at a time while carrying forward just enough memory of the past?

Recurrent neural networks were introduced as a direct answer to that question. A plain RNN keeps a hidden state, updates it whenever a new token arrives, and uses that state as a compressed summary of the past. A GRU keeps the same overall idea, but adds gates so that the model can decide what to retain and what to overwrite.

2 A Running Example

Throughout this note, imagine a sentiment classifier that reads one word at a time and must decide whether a review is positive or negative.

We compare two sentences:

``the movie is good'', ``the movie is not good''.

The final task is identical in both cases: output one label after reading the full sentence. The difficulty lies in memory. The model should not merely react to the latest word. It should build a small internal summary as the sentence unfolds.

For intuition, let the hidden state be a single real number:

- positive values suggest positive sentiment,
- negative values suggest negative sentiment,
- values near zero mean the model is still uncertain.

This scalar example is too small for real applications, but it is ideal for understanding the logic.

3 The Black-Box View

3.1 Input, Process, Output

Both RNN and GRU can be described as the same three-stage machine:

1. **Input.** Receive the next element x_t of the sequence.
2. **Process.** Combine x_t with a memory state h_{t-1} coming from the previous step.
3. **Output.** Produce a new memory state h_t, and possibly an external output $\hat{y}_t$ or a final prediction $\hat{y}$.

The only major difference is *how* the new memory is formed. A plain RNN uses one direct nonlinear update. A GRU uses gates to control memory flow.

3.2 Why This Is Different from an Ordinary Feedforward Network

In an ordinary feedforward network, the input is processed once and then discarded. In an RNN-like model, the hidden state becomes a moving summary:

> the old state tells the model what it already knows, and the new input tells the model what has just happened.

This is what makes recurrent models suitable for text, speech, time series, trajectories, and event streams.

4 The Plain RNN

4.1 Basic Mathematical Model

The standard plain RNN update is

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b),$$

where

- x_t is the input at time t,
- h_{t-1} is the previous hidden state,
- W_x and W_h are learned weight matrices,
- b is a bias vector,
- ϕ is usually tanh or another nonlinear function.

If we want a final classification score, we may use

$$\hat{y} = \sigma(W_y h_T + c),$$

after the last step T.

The meaning of the formula is straightforward. At time t, the model mixes two sources of information:

- what it sees now,
- what it remembers from before.

The output is a new summary of the prefix $(x_1, \ldots, x_t)$.

A compact picture makes the update rule easier to parse.

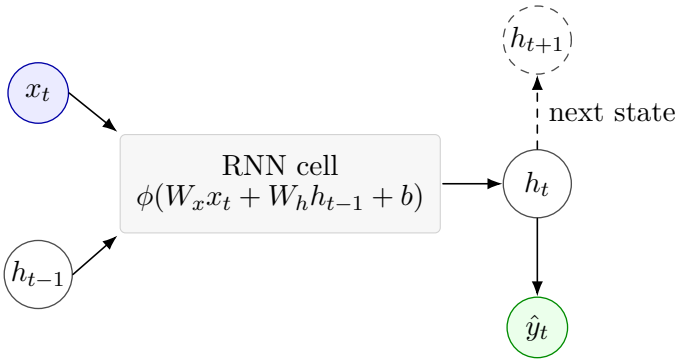

Figure 1: A plain RNN combines the current input with the previous hidden state to produce a new state, then optionally reads out a prediction.

4.2 A Naive Numerical Walkthrough

Let the hidden state be one-dimensional and use

$$h_t = \tanh(0.8x_t + 0.5h_{t-1}), \qquad h_0 = 0.$$

Assign simple scalar inputs:

$$x(\texttt{good}) = 1, \qquad x(\texttt{not}) = -1, \qquad x(\texttt{movie}) = x(\texttt{is}) = 0.$$

For the sentence

``movie is good'',

the updates are

$$h_1 = \tanh(0) = 0, \qquad h_2 = \tanh(0) = 0, \qquad h_3 = \tanh(0.8) \approx 0.66.$$

The final state is positive, which is sensible.

Now consider

``movie is not good''.

Then

$$h_1 = 0, \qquad h_2 = 0, \qquad h_3 = \tanh(-0.8) \approx -0.66.$$

At the final word `good`,

$$h_4 = \tanh(0.8 + 0.5(-0.66)) = \tanh(0.47) \approx 0.44.$$

This is the central weakness of a plain RNN. The last positive word pulls the state back toward positivity. The earlier negation is present, but only weakly. If the sentence were longer, that older signal could fade even further.

4.3 Algorithmic View

For a sequence $x_1, \ldots, x_T$, the plain RNN algorithm is:

1. Initialize h_0.
2. For $t = 1, \ldots, T$, compute $h_t = \phi(W_x x_t + W_h h_{t-1} + b)$.
3. Use h_T or all states (h_t) to produce the task output.

Training usually proceeds by backpropagation through time: unroll the recurrence across the sequence, compute the loss, and differentiate through all time steps.

5 Why the Plain RNN Struggles

5.1 The Intuition Behind Vanishing Memory

Each update compresses the old state into a new one. If that compression repeatedly shrinks useful signals, then distant information becomes hard to recover. One may summarize the problem in a single sentence:

> the model keeps rewriting its notebook, and after many rewrites the old sentence becomes too faint to read.

This is why plain RNNs often struggle with long-range dependence. They are elegant and conceptually important, but they do not protect memory very well.

5.2 Practical Consequences

In practice, a plain RNN may work reasonably when

- sequences are short,
- the important context is local,
- the computational budget is tight and a small model is acceptable.

It becomes less attractive when the task requires carrying a critical signal across many steps.

6 The GRU

6.1 Core Idea

The GRU, or Gated Recurrent Unit, keeps the recurrent structure but inserts gates. These gates are learned numbers between 0 and 1 that decide how strongly new information should enter and how strongly old information should remain.

The practical question answered by the GRU is not *"how do we create a hidden state?"* but rather *"how do we update memory without erasing useful history too quickly?"*

6.2 Basic Mathematical Model

A common GRU form is

$$
\begin{aligned}
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z), \\
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r), \\
\tilde{h}_t &= \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h), \\
h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t.
\end{aligned}
$$

Here

- z_t is the update gate,
- r_t is the reset gate,
- $\tilde{h}_t$ is the candidate state,

- $\odot$ denotes elementwise multiplication.

These equations look longer than the plain RNN equation, but the intuition is simple:

- the update gate decides whether to keep the old memory or replace it,
- the reset gate decides how much of the old state should influence the candidate memory.

The diagram below shows the same flow in one time step.

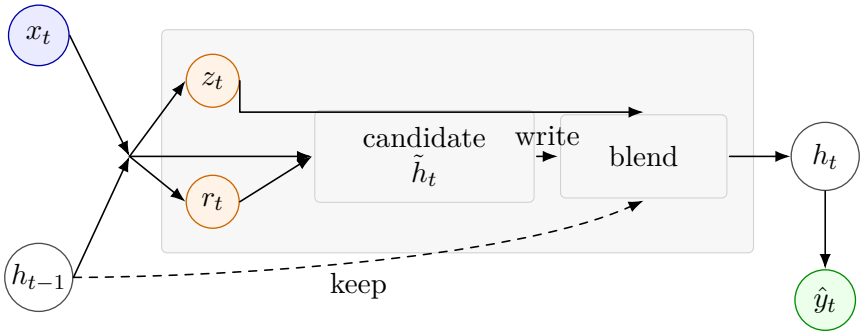

Figure 2: A GRU keeps the same recurrent backbone, but gates the flow of information so the old state is blended with a candidate update.

6.3 Intuitive Algorithm

At each time step, the GRU behaves like this:

1. Read the new input x_t.
2. Compute a tentative new memory $\tilde{h}_t$.
3. Ask how much of the old memory should be preserved.
4. Blend old memory and candidate memory to form h_t.

The blend is the key improvement. A plain RNN rewrites memory in one shot. A GRU performs a controlled overwrite.

6.4 A Naive Numerical Walkthrough

Return to the sentence

``movie is not good''.

Suppose the hidden state is again scalar and the network has already learned the following behavior:

- after reading `not`, it stores a strong negative memory,
- at filler words, it keeps most of that memory,
- at a sentiment word such as `good`, it mixes the new evidence with the old context instead of fully overwriting it.

Let

$$h_2 = 0, \qquad h_3 = -0.80$$

after reading `not`. Now assume the next word is a weak filler and the GRU chooses

$$z_4 = 0.10, \qquad \tilde{h}_4 = -0.40.$$

Then

$$h_4 = (1 - 0.10)(-0.80) + 0.10(-0.40) = -0.76.$$

So the negative memory is mostly preserved.

At the final word `good`, suppose the GRU computes

$$z_5 = 0.60, \qquad \tilde{h}_5 = 0.20.$$

Then

$$h_5 = 0.40(-0.76) + 0.60(0.20) = -0.184.$$

This final state is still slightly negative. That is much more plausible than the plain RNN output above. The positive word matters, but it does not erase the earlier negation.

7 RNN versus GRU

7.1 High-Level Comparison

Aspect	Plain RNN	GRU
Memory update	Direct overwrite by one nonlinear map	Gated blend of old and new memory
Parameter count	Smaller	Larger
Training stability	Often weaker on long sequences	Usually better
Ability to preserve context	Limited	Stronger
Interpretation	Simpler first model	More practical recurrent baseline

7.2 Trade-Offs

The plain RNN remains valuable because it is the cleanest recurrent idea. It is easy to teach, easy to implement, and small in parameter count. But that simplicity comes with fragile memory.

The GRU is usually the more practical choice when one wants a recurrent model and expects dependencies that are not purely local. The price is extra machinery and more parameters. One should therefore not think of the GRU as replacing the RNN conceptually. Rather, it is a repair to one of the RNN's main engineering weaknesses.

8 How the Neural Architecture Actually Learns

8.1 From Token to Prediction

It is useful to separate *the architecture* from *the optimizer*. The optimizer may be Adam or gradient descent, but what it is optimizing is a concrete pipeline:

1. a token or input value is converted into a vector representation,
2. the recurrent core mixes that new vector with the previous hidden state,
3. the final hidden state is mapped to a task output by a readout layer,
4. the loss compares that output with the target and sends gradients back through the whole chain.

For a text example, the architecture is not learning one mysterious object. It is learning several coordinated maps. The embedding layer learns how words should be represented as vectors. The recurrent core learns how new evidence should change memory. The readout layer learns which directions in hidden-state space matter for the final decision.

8.2 What Each Parameter Group Learns

Embedding layer. The embedding matrix learns which tokens should start near one another and which should be separated. In a sentiment task, the model may learn that `good` and `great` should point in related directions, while `not` should behave less like a sentiment word and more like a modifier that changes how later words are interpreted.

Recurrent core. The recurrent weights determine how past state and current input are fused. In a plain RNN, the same update mechanism must do many jobs at once:

- react to new information,
- preserve useful old information,
- forget irrelevant detail,
- and keep the dynamics numerically stable.

This is one reason the plain RNN can be hard to optimize.

In a GRU, the architecture gives the optimizer a more structured parameterization. The update gate learns when the old memory should mostly stay. The reset gate learns when the old memory should matter less while forming a candidate state. In other words, the architecture already separates several roles that the plain RNN tries to handle with one undifferentiated recurrence.

Readout layer. The readout layer learns how to turn hidden-state geometry into a task prediction. For binary sentiment classification, one may think of the readout as learning a direction in hidden-state space that means *"more positive"* versus *"more negative."* The hidden state need not look interpretable by itself; it only needs to become useful after the readout is applied.

8.3 What Backpropagation Through Time Is Actually Changing

The phrase *backpropagation through time* can sound abstract, so let us unpack it using the sentence

``` ``movie is not good''. ```

Suppose the model incorrectly predicts that the sentence is positive.
```

enter: locate the error. The output is wrong because the final hidden state was interpreted by the readout layer as too positive.

rw: send the error backward through the readout. The readout weights receive a gradient telling them that the final state should contribute less to the positive class.

rw: continue backward into the recurrent core. The recurrent parameters receive a gradient telling them that the effect of `not` was not preserved strongly enough until the final word `good` arrived.

rw: continue backward into the embedding layer. The embedding of `not` also receives a learning signal. The model is effectively told:

> this token should push the hidden state in a direction that survives long enough to change the later interpretation of `good`.

done: update all parameter groups together. After one optimizer step, the network has not merely *memorized a label*. It has slightly changed:

- the vector used for `not`,
- the recurrent rule that transports memory forward,
- and the readout rule that interprets the final state.

This is why architecture matters. Different parameterizations create different routes along which the same error signal can travel.

8.4 Why GRU Is Usually Easier to Learn

The GRU is not powerful because of magic. It is often easier to train because its architecture gives the optimizer a better search space.

In a plain RNN, one recurrent map must simultaneously decide

- how much new information to write,
- how much old information to keep,
- and how strongly gradients should survive over time.

In a GRU, these responsibilities are separated more cleanly. That does not guarantee better results on every small task. But it does mean that when the task truly requires long memory, the optimizer is less likely to fight against the architecture itself.

8.5 How This Connects to the Companion Code

This perspective also explains why the easy toy demo and the harder benchmark behave differently.

In the easy sentiment demo, the architecture does not need to learn a very delicate memory strategy. Short sequences and simple negation are often easy enough that both a plain RNN and a GRU can learn workable parameter settings.

In the long-memory benchmark, the architecture must learn something stricter:

write one bit early, protect it through many distractor steps, and still expose it at the end.

That is precisely the sort of task where the architectural separation inside the GRU becomes more valuable. So the benchmark is not only a story about accuracy; it is also a story about which neural architecture is easier to tune into the required behavior.

9 A Note on the Companion Code

The small sentiment demo that accompanies this note is intentionally easy. The dataset is tiny, the sentences are short, and the dependence pattern is simple. Under those conditions, a plain RNN and a GRU can look surprisingly similar, because both models can fit the task without being forced to manage truly long memory.

That is why the toy demo should be read as a mechanism demo, not as a decisive benchmark. It helps the reader see how recurrent state is updated step by step, but it does *not* by itself prove a large practical advantage for the GRU.

To reveal the engineering difference more honestly, one needs a harder benchmark: longer sequences, longer-distance dependencies, or evaluation beyond the training length. In that regime, the GRU often becomes more stable and more reliable. But even there, the correct claim is still modest:

> the GRU is often easier to train on harder memory problems, not guaranteed to dominate plain RNN on every small task.

Our current companion code now includes such a harder benchmark in `code/long_memory_benchmark`. The training setup is intentionally asymmetric: the models are trained only on short sequences of length 12, but then evaluated on much longer lengths up to 96. The task is to remember one bit written near the start of the sequence and reproduce it only after a long stream of distractor tokens.

In one representative run, the plain RNN achieved a mean accuracy of about 0.502 across the test lengths, while the GRU achieved about 1.000. The point of this number is not that GRU has won some universal contest. The point is narrower and more useful: once the task genuinely stresses long memory and length generalization, the engineering difference between direct recurrent overwrite and gated memory control becomes much easier to observe.

Model	Train length	Mean accuracy across test lengths
Plain RNN	12 only	about 0.502
GRU	12 only	about 1.000

9.1 A Precise Statement One Can Actually Prove

The sentence *"GRU converges whereas plain RNN does not converge"* is too strong to use as a theorem. In general, both model families may succeed or fail depending on parameters, initialization, optimization, and task design.

For the long-memory benchmark above, a mathematically defensible pair of statements is weaker and more precise:

1. a *contractive* plain RNN forgets early information exponentially fast,
2. a GRU admits parameter settings whose memory error can be made arbitrarily small on any fixed horizon.

Proposition 1 (contractive plain RNN forgets exponentially). Consider the scalar recurrence

$$h_t = \tanh(\alpha x_t + \beta h_{t-1}), \qquad |\beta| \le \rho < 1.$$

Take two sequences that are identical from time 2 onward and differ only in the stored bit at time 1. Let the corresponding hidden states be h_t^+ and h_t^-. Then

$$|h_T^+ - h_T^-| \le \rho^{T-1} |h_1^+ - h_1^-|.$$

Proof in small steps. **enter:** define the difference

$$\Delta_t := h_t^+ - h_t^-.$$

rw: because the two sequences share the same suffix, for every $t \ge 2$ they use the same input x_t. Hence

$$\Delta_t = \tanh(\alpha x_t + \beta h_{t-1}^+) - \tanh(\alpha x_t + \beta h_{t-1}^-).$$

have: the function tanh is 1-Lipschitz, so

$$|\tanh(u) - \tanh(v)| \le |u - v|.$$

Apply this with

$$u = \alpha x_t + \beta h_{t-1}^+, \qquad v = \alpha x_t + \beta h_{t-1}^-.$$

Then

$$|\Delta_t| \le |\beta|\,|h_{t-1}^+ - h_{t-1}^-| = |\beta|\,|\Delta_{t-1}| \le \rho |\Delta_{t-1}|.$$

done: iterate the same inequality from $t = 2$ to $t = T$:

$$|\Delta_T| \le \rho^{T-1} |\Delta_1|.$$

So the separation margin between two different stored bits decays exponentially with sequence length.

Interpretation. This does *not* prove that every plain RNN fails. It proves that once the recurrent map is contractive, early information is squeezed away exponentially fast. That is exactly the kind of mechanism that makes long length generalization fragile.

Proposition 2 (GRU admits arbitrarily accurate finite-horizon memory). Consider a scalar GRU state

$$h_t = (1 - z_t) h_{t-1} + z_t \tilde{h}_t, \qquad 0 \le z_t \le 1.$$

Fix a horizon T and a target tolerance $\delta > 0$. Suppose the stored bit is a target value $s \in [-m, m]$. If the parameters are chosen so that

$$|h_1 - s| \le 2m\varepsilon, \qquad z_t \le \varepsilon \text{ for } 2 \le t \le T, \qquad |\tilde{h}_t| \le m \text{ for } 2 \le t \le T,$$

then for all $1 \le t \le T$,

$$|h_t - s| \le 2mt\varepsilon.$$

Hence choosing

$$\varepsilon \le \frac{\delta}{2mT}$$

guarantees

$$|h_t - s| \le \delta \qquad \text{for all } 1 \le t \le T.$$

Proof in small steps. **enter:** define the memory error

$$e_t := |h_t - s|.$$

rw: substitute the GRU update:

$$e_t = \left|(1 - z_t)h_{t-1} + z_t\tilde{h}_t - s\right|.$$

Rewrite the right-hand side by adding and subtracting $(1 - z_t)s$:

$$e_t = \left|(1 - z_t)(h_{t-1} - s) + z_t(\tilde{h}_t - s)\right|.$$

have: apply the triangle inequality:

$$e_t \le (1 - z_t)|h_{t-1} - s| + z_t|\tilde{h}_t - s| = (1 - z_t)e_{t-1} + z_t|\tilde{h}_t - s|.$$

rw: because $|\tilde{h}_t| \le m$ and $|s| \le m$,

$$|\tilde{h}_t - s| \le |\tilde{h}_t| + |s| \le 2m.$$

Since $z_t \le \varepsilon$, we get

$$e_t \le e_{t-1} + 2m\varepsilon.$$

done: start from $e_1 \le 2m\varepsilon$, then iterate:

$$e_t \le e_1 + 2m(t - 1)\varepsilon \le 2mt\varepsilon.$$

Therefore choosing $\varepsilon \le \delta/(2mT)$ yields $e_t \le \delta$ for the whole horizon.

Why this matches the benchmark. The benchmark uses a finite alphabet of token types, so the GRU gates only need to behave correctly on finitely many input cases. On a finite set, sigmoid logits can be made arbitrarily negative on distractor tokens and strongly positive on write tokens. Therefore the hypothesis $z_t \le \varepsilon$ on distractors is not a fantasy; it is exactly the kind of gate pattern the GRU can learn to approximate.

Bottom line. The rigorous conclusion is therefore not *"GRU converges, RNN does not."* The rigorous conclusion is:

> for this kind of long-memory task, contractive plain RNNs forget early information exponentially fast, whereas GRUs admit near-identity memory transport over any fixed horizon.

That claim is mathematically defensible, and it is much closer to what the benchmark is actually showing.

10 A Unified State-Space View

10.1 Why the Comparison Is Reasonable

At a high enough level, GRU-like models and Kalman filtering do share a family resemblance. Both process a stream one step at a time. Both carry forward an internal state. Both update that state using new information that arrives at the current step. In that broad sense, both can be placed inside a common state-update picture:

$$\text{new state} = \text{update}(\text{old state}, \text{new input}).$$

This is the right unifying statement. The weaker statement *"both predict future actions"* is usually not correct. In many tasks, neither model is predicting an action at all. Rather, both are maintaining a useful internal description of what has happened so far.

10.2 How the Two State Updates Differ

For a Kalman filter, the internal state is an estimate of an underlying physical state, together with an uncertainty description. In a control setting, one often writes

$$\hat{x}_{t|t-1} = f(\hat{x}_{t-1|t-1}, u_{t-1}), \qquad \hat{x}_{t|t} = \text{correct}(\hat{x}_{t|t-1}, y_t),$$

where u_{t-1} is a control input and y_t is a new observation. The update is model-driven: the engineer specifies the dynamics and the measurement relation, and the filter uses them to estimate a latent state.

For a GRU, the internal state is a learned hidden representation. Its update has the schematic form

$$h_t = \text{GRUCell}(h_{t-1}, x_t),$$

where x_t is the next token, sensor reading, or time-series value. The update is data-driven: instead of being hand-specified from physics, it is learned from examples.

So the structural analogy is real:

- both are recursive state updaters,
- both compress the past into a current state,
- both use the current input to revise that state.

But the operational difference is equally important:

- a Kalman filter is an estimator with explicit uncertainty bookkeeping,
- a GRU is a learned memory machine whose hidden state usually has no direct physical meaning.

10.3 Connecting Back to the Running Example

In the sentiment example, the GRU hidden state stores a summary of the sentence prefix. After reading `not`, the state should carry a negative modifier forward so that the later word `good` is interpreted in context.

In a Kalman-filter example, one might track the position and velocity of a robot. The filter carries forward its best current estimate, then revises it after receiving a noisy sensor measurement. The role of memory is therefore similar: the present state summarizes relevant past information before the next piece of data arrives.

The difference is what that state means. In sentiment analysis, the hidden state is a useful learned summary for prediction. In robot tracking, the Kalman state aims to represent an external quantity that exists whether or not the estimator sees it perfectly.

10.4 The Practical Takeaway

If one wants a unified sentence, it is this:

> Kalman filtering and GRU both belong to the broader class of sequential state-update methods, but one is a model-based estimator with uncertainty, while the other is a data-driven learned hidden-state updater.

That viewpoint is useful because it prevents a false dichotomy. The two ideas are not identical, but they are not unrelated either. They sit on the same high-level ladder:

1. keep a state,
2. update it as new data arrives,
3. use the updated state for estimation, prediction, or decision.

11 Where RNN and GRU Are Actually Used

11.1 Typical Applications

These models appear naturally whenever data arrives in order:

- language modeling and sentiment analysis,
- speech and handwriting recognition,
- sensor forecasting and anomaly detection,
- medical time series,
- user behavior modeling.

11.2 A Simple Industrial Example

Imagine a factory machine with a temperature stream

$$x_1, x_2, \ldots, x_T.$$

The task is to predict whether the machine will overheat in the next minute.

A feedforward model that only looks at the current temperature may miss the trend. A recurrent model can remember that the temperature has been drifting upward for many steps. A GRU is especially useful if one short cooling event should not immediately erase the broader upward pattern.

12 What to Remember

The conceptual ladder is short:

1. A sequence task requires memory.
2. A plain RNN creates memory through repeated hidden-state updates.
3. A GRU keeps the same recurrent idea, but makes memory updates selective through gates.

If one sentence must summarize the difference, it is this:

> A plain RNN always rewrites its memory, while a GRU learns when to preserve, forget, and refresh it.

For a first encounter with sequence modeling, the RNN teaches the core idea. For many practical tasks, the GRU is the safer engineering choice.